

Programming Assignment 6

Filename: pa6.c

You must include the following commented lines at the beginning of your code to declare authorship. Submissions that do not include these lines will receive a 10% deduction.

```
/* COP 3502C PA6  
   This program is written by: Your Full Name */
```

Objective

In this assignment, you will practice:

- Array-based binary heap with mode-dependent comparator.
- Priority-based extraction using a unified service command.
- Key recomputation + $O(n)$ heapify on mode changes.
- Heap-based priority queue with linear search for name-based updates and removals.
- Clean dynamic memory management and deterministic output formatting.

Background Story

Welcome to *The Scratching Post*, the largest feline triage and adoption center in New York City. From sun-kissed strays to elderly long-timers, the center cares for thousands of cats each year.

But with rising rescue numbers, the staff can no longer manually decide: which cats need urgent medical care, which cats are ideal for same-day adoption, which cats should temporarily remain in quarantine, how shifting conditions (age, injuries, health changes) affect priority, and how special campaigns (like Featured Breed Week) influence adoptability.

They now turn to you, the center's most trusted systems engineer, to implement a dynamic, heap-driven priority engine capable of:

- Ranking cats in Adoption Mode (max-heap),
- Ranking cats in Triage Mode (min-heap),
- Automatically recalculating priorities when modes change,
- Supporting cat attribute updates,
- Enforcing Quarantine Protocols,
- Respecting Featured Breed bonuses,
- And handling all shelter operations through a consistent command language.

Your system must be fast, memory-safe, and perfectly deterministic, no surprises allowed when real cats' wellbeing is on the line.

System Overview

The system maintains a live collection of cats. For each cat, the system computes a priority key based on the current mode and stores the cat in a heap. Cats are identified by name using a linear search over the heap array when updates or removals are requested.

Two global modes exist:

1. **Adoption** - A max-heap ranking cats with the highest adoptability score at the top.
2. **Triage** - A min-heap ranking cats with the lowest (most urgent) triage score at the top.

Default mode is **Adoption**. Switching modes forces a full key recompute and heap rebuild. Following are the scoring formulae for the two global modes:

Mode 1: Adoption (Max-Heap)

$$\blacksquare \text{ base} = 1.6 \times \text{friendliness} + 1.1 \times \text{health} - 0.7 \times \text{age}$$

$$\blacksquare \text{ mult} = (\text{breed} == \text{featured?} : \alpha : 1.0)$$

$$\blacksquare \text{ adoption_key} = \text{base} \times \text{mult} + (-10^{-6} \times \text{arrival_id})$$

Higher is better. So, higher key ranks higher; tie-breakers $\Rightarrow$ lexicographically smaller name, then smaller *arrival_id*.

Mode 2: Triage (Min-Heap)

$$\blacksquare \text{ triage_key} = (100 - \text{health}) \times 2.0 + \max(0, \text{age} - 12) \times 1.0 - 0.05 \times \text{friendliness}$$

Lower = more urgent. So, lower key ranks higher; tie-breakers $\Rightarrow$ lexicographically smaller name, then smaller *arrival_id*.

Serving Cats

The system provides a unified **SERVE** command that removes and processes the highest-priority cat based on the current mode.

■ In **ADOPTION** mode, **SERVE** adopts the highest-priority non-quarantined cat (max-heap behavior).

■ In **TRIAGE** mode, **SERVE** treats the most urgent cat (min-heap behavior).

Switching modes changes the interpretation of priority but does not change the underlying heap structure.

Additionally, a **quarantine protocol** needs to be implemented. A quarantined cat (**quarantine** = 1) cannot be adopted. However, a quarantined cat can be treated in **TRIAGE** mode. In **ADOPTION** mode: the system extracts top cats until it finds a non-quarantined one; skipped quarantined cats are temporarily stored and reinserted afterwards. If only quarantined cats exist, the system will print: “No adoptable cats available.” Quarantine does not affect numeric priority values, only adoption eligibility.

Commands and Input Specifications

The first line contains q (number of commands). Then q lines of:

- **Add a cat:** ADD <name> <breed> <age> <friendliness> <health>
- **Update a cat:** UPDATE <name> <FIELD> <value>
- **Remove a cat:** REMOVE <name>
- **View top cat:** PEEK
- **Serve top-priority cat:** SERVE
- **Change mode:** MODE <ADOPTION|TRIAGE>
- **Set/reset featured breed:** FEATURED <breed> <alpha> or FEATURED NONE 1.0
- **Print top k (non-destructively):** PRINT <k>
- **Quit:** QUIT

Required Data Structures and Function Prototypes

Do not rename, remove, or alter any required fields of the provided structures. You may add additional fields or helper structs only if they don't break these definitions or the specified behaviors. The provided function prototypes are also mandatory. You may write additional helper functions, but do not alter or overload these required prototypes.

```
/* ----- Global Modes ----- */
typedef enum {
    MODE_ADOPTION = 0,      /* Max-heap by adoption_key (higher is better)
    */
    MODE_TRIAGE    = 1      /* Min-heap by triage_key    (lower is more
    urgent)*/
} Mode;

/* ----- Core Cat Record ----- */
typedef struct Cat {
    char *name;              /* dynamically allocated, unique string (<= 25
    chars + '\0') */
    char *breed;             /* dynamically allocated string */
    int age;                 /* years */
    int friendliness;        /* 0..100 */
    int health;              /* 0..100      (higher means healthier) */
    int arrival_id;          /* strictly increasing when ADDED */
    int quarantine;          /* 0 or 1 (1 => cannot be adopted; S1) */
    double key;              /* cached priority value for the **active**
    mode */
} Cat;

/* ----- Array-Based Binary Heap of Cat* ----- */
typedef struct {
```

```

    Cat **arr;           /* array of Cat* implementing the heap */
    int size;            /* current number of elements */
    int capacity;        /* allocated capacity */
    Mode mode;           /* controls comparator semantics */
} CatHeap;

/* ----- Global Shelter State ----- */
typedef struct {
    Mode mode;           /* MODE_ADOPTION or MODE_TRIAGE */
    char *featured_breed; /* NULL => none; else dynamically allocated
                           breed string */
    double alpha;        /* multiplier for featured breed (>= 1.0;
                           default 1.0)*/
    int next_arrival_id; /* increments on each ADD */
    CatHeap heap;        /* priority structure */
} Shelter;

/* Returns heap index of cat with given name, or -1 if not found */
int find_cat_index(const CatHeap *heap, const char *name);
/* Returns the current adoption-mode key for cat c using S->featured_breed
   and S->alpha. */
double compute_adoption_key(const Cat *c, const Shelter *S);
/* Returns the current triage-mode key for cat c. */
double compute_triage_key(const Cat *c);
/* Recomputes all keys for the active mode and rebuilds heap in O(n) using
   bottom-up heapify. */
void recompute_all_keys_and_build(Shelter *S);

/* ===== Command Handlers (I/O-Free Logic) ===== */
/* Allocates a new Cat, initializes fields, computes key for active mode,
   ensures no duplicate name exists (linear scan), and inserts into the
   heap. */
void cmd_add(Shelter *S, const char *name, const char *breed, int age, int
    friendl, int health);
/* Locate the cat by name using a linear scan of the heap array.
   If found, update the requested field.
   For AGE/FRIEND/HEALTH: recompute key for active mode and restore heap
   order.
   For QUARANTINE: update the flag only (numeric key unchanged), then
   restore heap order. */
void cmd_update(Shelter *S, const char *name, const char *field, int
    new_value);
/* Locate the cat by name using a linear scan of the heap array.
   Remove it from the heap, restore heap order, and free the Cat. */
void cmd_remove(Shelter *S, const char *name);
/* Prints the current top cat for the active mode (does not modify heap).
   */
void cmd_peek(const Shelter *S);
/* Serves the highest-priority cat based on the active mode.
   ADOPTION: adopt highest-priority non-quarantined cat.
   TRIAGE: treat most urgent cat. */
void cmd_serve(Shelter *S);

```

```
/* Sets S->mode, sets heap mode, then recompute_all_keys_and_build(S). */  
void cmd_mode(Shelter *S, const char *mode_str);  
/* Sets (or clears) featured breed and alpha, then  
   recompute_all_keys_and_build(S). */  
void cmd_featured(Shelter *S, const char *breed, double alpha);  
/* Non-destructive: print top k according to active mode.  
   Recommended: copy heap array into a temp heap and extract k from the  
   copy. */  
void cmd_print(const Shelter *S, int k);
```

Output Specifications

General rules (apply to every line you print)

- Exact text/case/spacing; ASCII; no extra blank lines.
- Keys and alpha: two decimals (rounded).
- Follow per-command patterns as shown.

1. **Mode Changes, print:** Mode set to <MODE>. Rebuilding priorities...
Only ADOPTION and TRIAGE are valid modes. For all other entries, print:
Unknown mode <unknown>.

2. Featured Breed:

- **Set or change to a breed, print:**
Featured breed set to <breed> with alpha=<alpha_formatted>.
Rebuilding priorities...
- **Clear featured breed, print:** Featured breed cleared.
Rebuilding priorities...

3. Adding Cats:

- On success, print: Added <name>.
- If a cat with the same name already exists, print: Name <name> already exists.

Note: ADD never prints keys or mode tags.

4. Peek Top Cat:

- If the heap is empty, print: No cats available.
- Otherwise, print one line describing the current top for the active mode:
Top[<MODE>]: [<MODE>] (key=XX.YY, name=<name>, breed=<breed>,
age=<age>, friend=<friendliness>, health=<health>)

Formatting details: Exactly one space after the colon :. The inner tag [ADOPTION] or [TRIAGE] must match the mode in the brackets after Top[...]. Key is two decimals.

5. Serve Cat (SERVE):

If the heap is empty, print: No cats available.

If current mode is ADOPTION:

- If all cats are quarantined, print: No adoptable cats available.
- Otherwise, the highest-priority non-quarantined cat is adopted, print:
Serve now: <name> (key=XX.YY, name=<name>, breed=<breed>,
age=<age>, friend=<friendliness>, health=<health>)

If current mode is TRIAGE:

- The most urgent cat is treated, print:
Serve now: <name> (key=XX.YY, name=<name>, breed=<breed>,
age=<age>, friend=<friendliness>, health=<health>)

Note: Only one cat is served per `SERVE` command. Quarantine blocks adoption but does not affect `TRIAGE`.

6. Update Cat:

- If `<name>` doesn't exist, print: `Cat <name> not found`.
- For `FIELD` in `{AGE, FRIEND, HEALTH}`:
Update the field, recompute key, adjust heap position, then print:
`Updated <name>: <FIELD>=<value>. Priority adjusted`.
- For `FIELD == QUARANTINE`:
Accept 0 or 1; adjust position (safe no-op) and print:
`Updated <name>: QUARANTINE=<0_or_1>`.
- For any other field, print: `Unknown field <FIELD>`.

Note: Breed is immutable after `ADD`; `UPDATE` does not support changing breeds.

7. Remove Cat:

- If `<name>` doesn't exist, print: `Cat <name> not found`.
- On success, print: `Removed <name>`.

8. Print Top-k (Non-Destructive):

- If heap is empty, print: `No cats available`.
- Otherwise, print up to `k` lines (or fewer if fewer than `k` cats exist):
`[<rank>] <name> (key=XX.YY, <MODE>)`

Formatting details: Square brackets around the 1-based rank. One space after the closing bracket. Exactly one space after the comma before the mode label. Do not include breed/age/friend/health here.

9. Quit: Print nothing for `QUIT`. Program terminates after processing `QUIT` or after reading all `q` commands.

10. Formatting & Rounding Rules:

- **Priority keys (key=XX.YY) and Alpha:** Always two decimals, rounded using standard rounding (e.g., $1.234 \rightarrow 1.23$, $1.235 \rightarrow 1.24$). No thousand separators.
- **Integers (age, friend, health, counts, ranks, total):** Print as plain base-10 integers; no leading zeros.
- **Spacing:** Exactly one space after colons `:` and commas `,`, where shown. Do not add trailing spaces at the end of lines.

Runtime and Memory Requirements

Runtime:

- Insert: $O(\log n)$
- Update by name: $O(n)$

- Remove by name: $O(n)$
- Serve: $O(\log n)$
- Mode / Featured change: $O(n)$
- PRINT k: $O(k \log n)$ using a temporary heap
- $n \leq$ approximately 5,000 cats

Memory:

- All strings dynamically allocated
- Free all dynamically allocated memory (cats, strings, heap, featured breed)
- No memory leaks permitted

Sample I/O

```

Sample Input:
13
MODE ADOPTION
ADD Coco Ragdoll 4 88 90
ADD Hana DSH 3 92 85
UPDATE Coco QUARANTINE 1
UPDATE Hana QUARANTINE 1
SERVE
UPDATE Hana QUARANTINE 0
SERVE
MODE TRIAGE
ADD Milo Siamese 15 40 60
SERVE
PRINT 2
QUIT
Sample Output:
Mode set to ADOPTION. Rebuilding priorities...
Added Coco.
Added Hana.
Updated Coco: QUARANTINE=1.
Updated Hana: QUARANTINE=1.
No adoptable cats available.
Updated Hana: QUARANTINE=0.
Serve now: Hana (key=238.60, name=Hana, breed=DSH, age=3, friend=92, health=85)
Mode set to TRIAGE. Rebuilding priorities...
Added Milo.
Serve now: Coco (key=15.60, name=Coco, breed=Ragdoll, age=4, friend=88, health=90)
[1] Milo (key=81.00, TRIAGE)

```

Implementation Hint

- You are NOT required to use any tree or map structure.
- To find a cat by name, simply scan the heap array.
- Focus on correctly restoring the heap after updates or removals.

Implementation Requirements

- Your program must be written in C and must compile on Gradescope.
- Make sure to submit well-structured and well-commented code to avoid penalties.
- No global variables except those explicitly required.
- You may not use C++ features, external libraries beyond standard C libraries such as `qsort`, `priority_queue`, `map`, etc., `system()` calls, or any OS-specific calls. Such approach will result in a **-100 penalty**.
- You may not use multiple `.c` or `.h` files, preprocessor tricks that alter required structures, or hard-coded output for any scenario.
- All output must be printed using `printf` exactly as specified in the “Output Specifications” section. No additional or missing newlines.
- All cat names and breed strings must be read into a temporary static buffer first, then dynamically allocated with the exact amount of memory needed, and finally copied using `strcpy`.
- All memory must be dynamically allocated using `malloc`, `calloc`, or `realloc`.
- You must not: Use global static arrays, Use fixed-length arrays for cat storage, Leak memory in any scenario.
- At the end of execution (before quitting), you must free: Every remaining cat, All name and breed strings, everything that had memory allocated. When removing a cat, you must free: `cat->name`, `cat->breed`, and the `Cat*` object itself.